

# De HTR au TALN

*Chapitre 2 — Ingestion du « data contract »*

# Pourquoi un chapitre entier sur les données ?

*Comprendre ses données n'est pas préparatoire : c'est le travail réel*

- Erreur fréquente : entraîner un modèle sur des données qu'on n'a pas comprises
- Objectif mesurable : connaître le taux de needs\_review, la couverture CamemBERT, et l'inventaire des abréviations de votre corpus
- Sans cette base, les ablations du Jour 2 seront ininterprétables

*Une ingénierie NLP rigoureuse se distingue par sa compréhension des données — pas seulement par ses expériences de modélisation.*

# Plan du chapitre

- 1. Le « data contract » HTR -> TALN : structure et garanties
- 2. Analyse des lignes needs\_review
- 3. Analyse exploratoire du corpus (EDA)
- 4. Inventaire des abréviations non résolues
- 5. Tokenisation et couverture de vocabulaire
- 6. Visualisation des embeddings avec UMAP
- 7. Verrouillage du split train/val/test
- 8. Récapitulatif : les chiffres à connaître avant le Jour 2



## PARTIE 1

# Le data contract HTR > TALN : structure et garanties

# Qu'est-ce qu'un data contract ?

*Le contrat d'entrée entre le Volet 1 (HTR) et le Volet 2 (NLP)*

- Un accord formel sur le format, la sémantique et les garanties de qualité des données échangées
- Le Volet 1 (Computer Vision / HTR) produit un fichier JSON consommé par le module TALN
- Ce contrat spécifie aussi des métriques de qualité : CER estimé  $\leq 12\%$ , lignes needs\_review, scores de confiance par caractère

*La première étape n'est pas de tokeniser : c'est de lire le schéma, valider sa conformité, et comprendre ce que chaque champ signifie opérationnellement.*

# Anatomie du JSON

*Structure hiérarchique du data contract*

```
document
├── document_id      : "charte_normandie_0042"
├── document_type    : charte | registre | roman | liturgique
├── metadata
│   ├── htr_model, cer_estimate, total_lines, needs_review_count
├── pages[]
│   └── lines[]
│       ├── line_id, transcription, confidence, needs_review
│       ├── char_confidences : scores par caractère
│       ├── candidates      : alternatives HTR par position
│       └── polygon         : coordonnées sur l'image source
```



# Signification opérationnelle des champs clés

## confidence & char\_confidences

Moyenne géométrique des confiances par caractère. Un score faible ne veut pas dire "faux" — juste "incertain". Servira de poids de perte au Jour 2.

## candidates

Positions où le modèle HTR a hésité entre deux caractères (ex. ~ vs a). La notation [a/b] du syllabus correspond à ce champ.

## needs\_review & polygon

Flag agrégé (seuil 0.75 typiquement) à décomposer. polygon ancre la ligne à l'image — utile au Jour 4 pour le graphe de connaissances.



# Validation du schéma en Python

*Un data contract sans validation est une source de bugs silencieux*

```
SCHEMA = {  
    "type": "object",  
    "required": ["document_id", "document_type", "metadata", "pages"],  
    "properties": {  
        "document_type": {"enum": ["charte", "registre", "roman", "liturgique"]},  
        "metadata": {"required": ["htr_model", "cer_estimate", "total_lines"]},  
        "pages": {"type": "array", "items": {"required": ["lines"]}}  
    }  
}
```

```
jsonschema.validate(doc, SCHEMA) # lève ValidationError si invalide
```

*Chargez l'ensemble du corpus (plusieurs fichiers JSON), validez chaque document, et aplatissez les lignes avec leur contexte (document\_id, document\_type, page\_id) pour l'analyse.*



PARTIE 2

# Analyse des lignes needs\_review

# Pourquoi cette analyse est non négociable

*needs\_review : votre première ligne de défense*

- Inclure ces lignes sans stratégie = entraîner sur du bruit non contrôlé
- Les exclure systématiquement = perdre des exemples rares, surtout si certains types de documents ont un taux structurellement plus élevé

*La bonne approche : comprendre la distribution de ces lignes avant de décider de leur sort.*

# Taux de needs\_review par type de document

*Résultat typique sur un corpus CREMMA équilibré*

| document_type | n_review | n_total | taux_pct |
|---------------|----------|---------|----------|
| roman         | 412      | 1203    | 34.2     |
| registre      | 287      | 891     | 32.2     |
| charte        | 156      | 743     | 21.0     |
| liturgique    | 41       | 312     | 13.1     |

*Les romans médiévaux ont un taux needs\_review 2.6 fois supérieur aux textes liturgiques — sans doute des scripts plus variables ou des abréviations plus denses. Pour la NER du Jour 3 : les entités des romans seront moins fiables, l'évaluation doit en tenir compte.*



# Décomposer les causes du signalement

*needs\_review agrège plusieurs conditions*

```
def diagnose_line(line):
    causes = []
    if line["confidence"] < 0.75:
        causes.append("confidence_globale_basse")
    if min(line["char_confidences"]) < 0.5:
        causes.append("caractere_tres_incertain")
    if len(line["candidates"]) > 0:
        causes.append("candidats_alternatifs")
    if re.search(r'[~ ñ]|[a-z]\^', line["transcription"]):
        causes.append("abreviation_residuelle")
    if len(line["transcription"].split()) < 3:
        causes.append("ligne_trop_courte")
    return causes or ["cause_inconnue"]
```

# Quatre stratégies de gestion

*Elles ne s'excluent pas mutuellement*

## Exclusion totale

Retirer toutes les lignes needs\_review. Simple, traçable. Risque : biais de sélection.

## Pondération différentielle

Inclure avec un poids de perte réduit, proportionnel à la confiance. Approche recommandée.

## Révision manuelle prioritaire

Réservée aux lignes confidence > 0.65, proches du seuil et récupérables.

## Ensemble séparé

Utiliser ces lignes comme validation de robustesse aux entrées bruitées.

*Quelle que soit la stratégie retenue, elle doit être documentée dans CONVENTIONS\_NLP.md avec son justificatif — un choix non documenté est un biais non traçable.*

PARTIE 3

# Analyse exploratoire du corpus (EDA)



# Distribution des longueurs de lignes

*Conditionne SEQ\_LEN, troncature et padding*

- Histogramme des longueurs en tokens, avec médiane et P95 marqués
- Comparaison de la distribution par type de document

*Ce qui compte : le P95 en tokens BPE (pas en mots bruts — les abréviations fragmentées augmentent ce compte). Si  $P95 > 128$ , il faudra tronquer ou segmenter. Pour BERT (512 tokens), généralement pas un problème au niveau ligne — mais ça le devient en agrégeant des lignes en paragraphes (Jour 4).*

```
df['n_tokens_brut'] = df['transcription'].apply(lambda t: len(t.split()))
```



# Distribution des scores de confiance HTR

*Confiance par ligne et par caractère*

- Histogramme global des confiances de ligne, avec le seuil needs\_review (0.75) marqué
- Histogramme des char\_confidences (toutes positions), avec seuils critique (0.5) et attention (0.7)

*Interprétation attendue : distribution bimodale — une masse autour de 0.95–0.99 (caractères bien reconnus), et une queue à gauche autour de 0.5–0.65 (abréviations, lettres inhabituelles, encre dégradée).*

*Ce sont les caractères de la queue gauche qui constituent votre problème de normalisation au Jour 2.*

# Mesurer les caractères critiques

*Deux seuils à retenir*

```
all_char_confs = [c for confs in df["char_confidences"] for c in confs]

n_critique = sum(1 for c in all_char_confs if c < 0.5)
print(f"Caractères avec confiance < 0.5 : {n_critique/len(all_char_confs)*100:.1f} %")

n_attention = sum(1 for c in all_char_confs if c < 0.7)
print(f"Caractères avec confiance < 0.7 : {n_attention/len(all_char_confs)*100:.1f} %")
```

*Ces deux pourcentages quantifient l'ampleur du problème de normalisation avant même d'avoir écrit une seule règle.*

## **PARTIE 4**

# **Inventaire des abréviations non résolues**



# Quatre formes d'abréviations médiévales

*L'analyse la plus directement opérationnelle de ce chapitre*

## Tilde de nasalité

norm<sup>~</sup>die (normandie), co<sup>~</sup>te (conte), q<sup>~</sup> (que)

## Lettre superscrite

p<sup>ñ</sup> (prison), p<sup>ñ</sup>ce (prince)

## Signes suprascripts

mp (monsieur), sr (seigneur)

## Lettres suspendues

p barré (par/per/pro), q barré (que/qui)

# Détection par expressions régulières

## *Patterns d'abréviations médiévales*

```
ABBREV_PATTERNS = {  
    "tilde_nasale": r'\w+\~\w*',      # co~te, norm~die, q~  
    "lettre_suprascr": r'[a-z]\^[a-z]?', # m^r, s^r  
    "lettre_speciale": r'[ñ]',        # ñ comme marque d'abréviation  
    "barre_p":      r'\bṑ\b|\bṕ\b',  # p barré unicode médiéval  
    "tilde_seul":   r'\s~\s|^~|^~$',  # tilde isolé  
}  
  
# Inventaire global + par type de document  
abbrev_inventory = Counter()  
abbrev_by_type   = {t: Counter() for t in df["document_type"].unique()}
```

# Le indicateurs de base

*Combien d'abréviations devez-vous résoudre ?*

- Total d'abréviations non résolues, et nombre de lignes avec  $\geq 1$  abréviation
- Répartition par patron (tilde\_nasale, lettre\_suprascr, ...) et par type de document

*Exemple : si votre corpus contient 847 tildes de nasalité, vous saurez que votre module de règles du Jour 2 devra couvrir ce cas — et vous mesurerez combien il en résout effectivement.*

*Livrable attendu : **abbreviations\_inventory.json** — table comptée par type de document et par patron, plus la couverture de la table statique mesurée.*



# Table d'abréviations contextuelles

*Certaines abréviations se résolvent sans modèle*

```
ABBREV_TABLE = {  
    "q~": "que", "Q~": "Que", "no~": "nom",  
    "sr": "seigneur", "S^r": "Seigneur", "m^e": "messire",  
    "norm~die": "normandie", "co~te": "conte", "champ~e": "champagne",  
    "l~": "livres", "s~": "sous",  
}  
  
# Couverture de la table sur le corpus  
resolvable = sum(1 for ... if abbrev in ABBREV_TABLE)  
print(f"Couverture table statique : {resolvable}/{total_tildes} "  
      f"({resolvable/total_tildes*100:.1f} %)")
```



## PARTIE 5

# Tokenisation et couverture de vocabulaire

# Le problème de fond

*CamemBERT : 32 000 tokens optimisés pour le français moderne*

- Les graphies médiévales (roys, palays, chastelain) ne correspondent pas aux formes modernes
- Les abréviations manuscrites (norm~die, co~te) sont entièrement hors-vocabulaire
- Les caractères spéciaux médiévaux peuvent être absents du vocabulaire Unicode couvert

*Conséquence : là où CamemBERT encode roi en un seul token, il encode roys en deux ou trois sous-mots — fragmentation accrue, qualité de représentation dégradée.*

# Deux indicateurs : OOV et fragmentation

*measure\_coverage(texts, tokenizer)*

## Taux OOV

Proportion de tokens résultant d'un [UNK]  
(vocabulaire absent)

## Fragmentation

Ratio (nb tokens BPE) / (nb mots bruts). 1.0 = pas de fragmentation ; 2.0 = chaque mot découpé en 2 sous-mots en moyenne

```
for text in texts:
    ids = tokenizer.encode(text, add_special_tokens=False)
    total_bpe_tokens += len(ids)
    total_unk_tokens += sum(1 for i in ids if i == unk_token_id)

taux_oov = total_unk_tokens / total_bpe_tokens
fragmentation = total_bpe_tokens / total_words
```



# Résultats attendus sur CREMMA Medieval

*Comparaison CamemBERT vs mBERT multilingue*

| Tokeniseur           | OOV global | Fragmentation<br>(propre) | Fragmentation<br>(needs_review) |
|----------------------|------------|---------------------------|---------------------------------|
| CamemBERT            | 0.3-2 %    | 1.4-1.8                   | 2.1-2.8                         |
| mBERT (multilingual) | 1-4 %      | 1.6-2.0                   | 2.4-3.2                         |

*Le taux OOV absolu reste faible : SentencePiece peut toujours fragmenter en caractères individuels. L'indicateur sensible, c'est la fragmentation.*

*Un facteur de 2.5 sur les lignes needs\_review signifie que chaque mot brut génère 2.5 tokens BPE en moyenne : CamemBERT "voit" ces mots comme des suites de sous-mots inconnus.*

# Identifier les formes les plus fragmentées

*word\_fragmentation\_analysis — ratio tokens/mot*

```
for word in text.split():
    tokens = tokenizer.tokenize(word)
    word_stats[word]["total_tokens"] += len(tokens)

avg_tokens = total_tokens / count
# Tri décroissant -> mots les plus pénalisés par le BPE

# Exemple de sortie :
# chastelain -> ['_ch','as','tel','ain'] (avg=4.0)
# roys       -> ['_ro','ys']           (avg=2.0)
```

*Ces formes sont précisément celles que la normalisation du Jour 2 devra traiter en priorité : une fois normalisées (chastelain -> châtelain, roys -> roi), leur fragmentation diminue.*

# Alternatives à la tokenisation BPE

*Character-level, ByT5, ou tokeniseur ré-entraîné*

## Character-level

Aucun OOV, couverture parfaite.  
Mais séquences 4-6× plus longues  
(coût quadratique) et perte des  
unités morphologiques.

## ByT5 (byte-level)

T5 sur octets UTF-8. Pas de  
tokeniseur, couverture universelle,  
mais séquences encore plus  
longues.

## Tokeniseur ré-entraîné

SentencePiece sur votre corpus, si  
> 10 000 lignes — sinon  
vocabulaire insuffisamment  
représentatif.

*Recommandation pour ce module : CamemBERT reste le choix par défaut — poids pré-entraînés utiles même pour le moyen français, normalisation du Jour 2 réduisant la fragmentation. Mais le coût de ce choix doit être mesuré avant d'être assumé.*



PARTIE 6

# Visualisation des embeddings avec UMAP



# Objectif de la visualisation

*UMAP : réduction de dimension non-linéaire (McInnes et al., 2018)*

- Vérifier que les types de documents forment des clusters distincts — CamemBERT capte-t-il des différences stylistiques entre chartes, registres, romans ?
- Identifier des anomalies : lignes aberrantes, documents mal classifiés, zones de chevauchement de genre
- Visualiser la séparation lignes propres / needs\_review dans l'espace d'embedding

*UMAP préserve mieux la structure locale que t-SNE pour des corpus de taille intermédiaire.*

# Extraire les embeddings [CLS]

*Le token [CLS] agrège le contexte de toute la séquence*

```
tokenizer = AutoTokenizer.from_pretrained("almanach/camembert-base")
model = AutoModel.from_pretrained("almanach/camembert-base")

inputs = tokenizer(batch, return_tensors="pt", padding=True,
                    truncation=True, max_length=128)
outputs = model(**inputs)

# outputs.last_hidden_state : (batch, seq_len, hidden_size)
cls_emb = outputs.last_hidden_state[:, 0, :].cpu().numpy()

# Échantillon de 500 lignes (UMAP > 5000 lignes est lent sur CPU)
```

# Réduction UMAP

## *Paramètres clés*

```
reducer = umap.UMAP(  
    n_components=2,  
    n_neighbors=15, # compromis local / global  
    min_dist=0.1,   # compacité des clusters projetés  
    metric="cosine", # standard pour les embeddings de langue  
    random_state=42,  
)  
embedding_2d = reducer.fit_transform(embeddings)  
  
# Deux scatter plots :  
# - colorés par document_type  
# - colorés par needs_review (rouge) vs propre (bleu)
```



# Interprétation et point de vigilance

## Clusters par type de document

Si les types forment des clusters relativement séparés, CamemBERT capte des différences stylistiques entre genres malgré la fragmentation BPE.

## needs\_review dans l'espace d'embedding

Si needs\_review est distribué uniformément (et non concentré dans une région), le flag ne correspond pas à un type linguistique particulier — ce qui est plutôt rassurant.

*Point de vigilance : UMAP préserve la structure locale mais pas nécessairement la globale. Des clusters visuellement séparés peuvent être plus proches en 768D qu'ils ne le semblent en 2D — ne sur-interprétez pas les distances inter-clusters.*

## PARTIE 7

# Verrouillage du split « train | val | test »

# Pourquoi verrouiller avant toute analyse

*Le split doit être fixé avant l'EDA approfondie*

- Si vous construisez un split "équilibré" après avoir analysé les distributions, vous introduisez un biais de sélection
- Le jeu de test deviendrait proche du jeu d'entraînement par construction → métriques optimistes

*La règle : fixer le split sur un critère indépendant du contenu (l'identifiant de document), et le verrouiller avec un hachage SHA-256.*



# Deux contraintes pour les corpus médiévaux

*Pas de fuite de document + stratification par genre*

## Pas de fuite de document

Toutes les lignes d'un même document doivent être dans le même split. Sinon, train et test partagent la même main, le même scribe, le même style — le modèle mémorise plutôt que de généraliser.

## Stratification par type

Si le corpus contient 70% chartes, 20% registres, 10% romans, chaque split doit respecter approximativement cette distribution.

```
TRAIN_RATIO, VAL_RATIO, TEST_RATIO = 0.70, 0.15, 0.15
```

```
# GroupShuffleSplit au niveau document_id, stratifié par document_type  
splitter = GroupShuffleSplit(n_splits=1, test_size=TEST_RATIO, random_state=42)  
train_val_idx, test_idx = next(splitter.split(doc_df, groups=doc_df["document_type"]))
```

```
# Vérification : pas de chevauchement entre train / val / test
```



# Verrouillage par SHA-256

*Le manifeste de split — règle absolue*

```
split_manifest = sorted(zip(df["line_id"].tolist(), df["split"].tolist()))
manifest_str = json.dumps(split_manifest, ensure_ascii=False, sort_keys=True)
SPLIT_HASH = hashlib.sha256(manifest_str.encode("utf-8")).hexdigest()

# Sauvegarder : split_manifest.json
# hash_sha256, train/val/test_doc_ids,
# n_train_lines, n_val_lines, n_test_lines, ratios
```

*Une fois split\_manifest.json versionné dans git et partagé, il ne doit plus être modifié. Toute modification invalide toutes les comparaisons de métriques antérieures.*

*Si le split doit changer (corpus élargi, erreur de stratification), créez un nouveau manifeste avec un suffixe de version et documentez la raison dans CONVENTIONS\_NLP.md.*

PARTIE 8

# Synthèse

# Le tableau à remplir sans consulter vos notebooks

| Métrique                                              | Valeur mesurée |
|-------------------------------------------------------|----------------|
| Nombre total de lignes                                |                |
| Taux de lignes needs_review global                    |                |
| Taux de needs_review par type de document (4 valeurs) |                |
| Cause principale du flag needs_review                 |                |
| Nombre d'abréviations non résolues (tildes)           |                |
| Couverture de la table statique d'abréviations        |                |
| Facteur de fragmentation CamemBERT (lignes propres)   |                |
| Facteur de fragmentation CamemBERT (needs_review)     |                |
| Longueur médiane en tokens BPE                        |                |
| P95 de longueur en tokens BPE                         |                |
| Hash SHA-256 du split                                 |                |



# Ce qu'il faut retenir

- Le data contract HTR -> TALN n'est pas un détail de format : ses champs (confidence, candidates, needs\_review) portent une sémantique opérationnelle à exploiter
- needs\_review doit être décomposé par cause et par type de document avant toute décision de filtrage
- L'inventaire des abréviations et la mesure de fragmentation CamemBERT constituent votre baseline pour le Jour 2
- UMAP aide à valider que les embeddings capturent une structure stylistique cohérente — sans sur-interpréter les distances
- Le split train/val/test doit être verrouillé par hash SHA-256 avant toute analyse approfondie, pour éviter tout biais de sélection